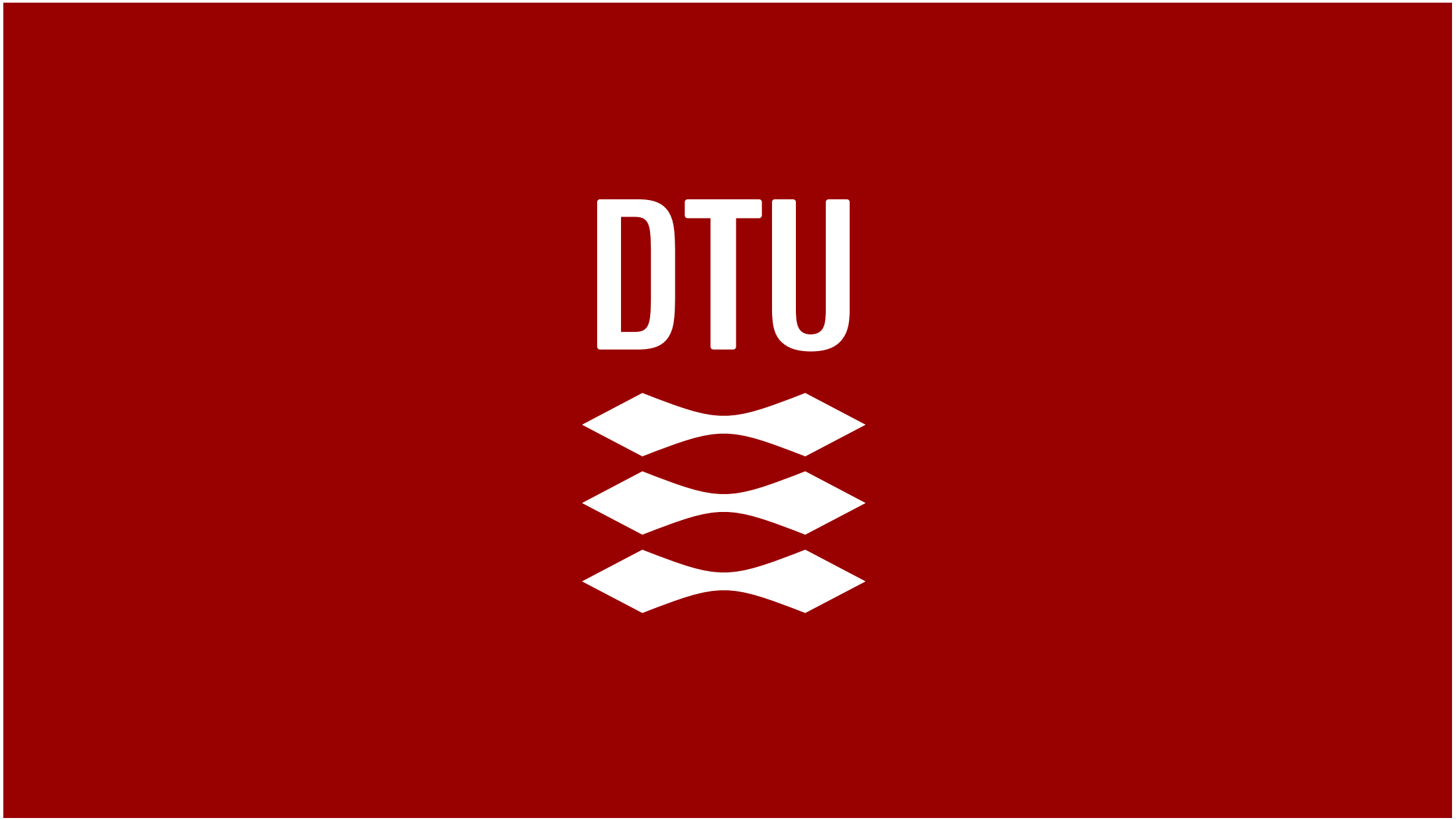

DTU

02159 – Operating Systems

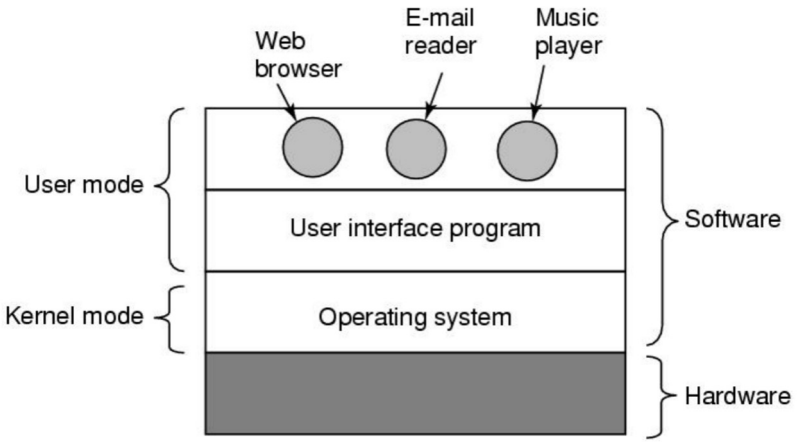
Week 4: OS Concepts and System Calls

Xenofon (Fontas) Fafoutis
Professor
xefa@dtu.dk
www.compute.dtu.dk/~xefa

Modes of Operation

- Kernel mode
  - **All** instructions and all hardware resources are accessible
- User mode
  - Accessible instructions and hardware resources are **limited**

- System call
  - To access these protected resources the programme must make a **system call**

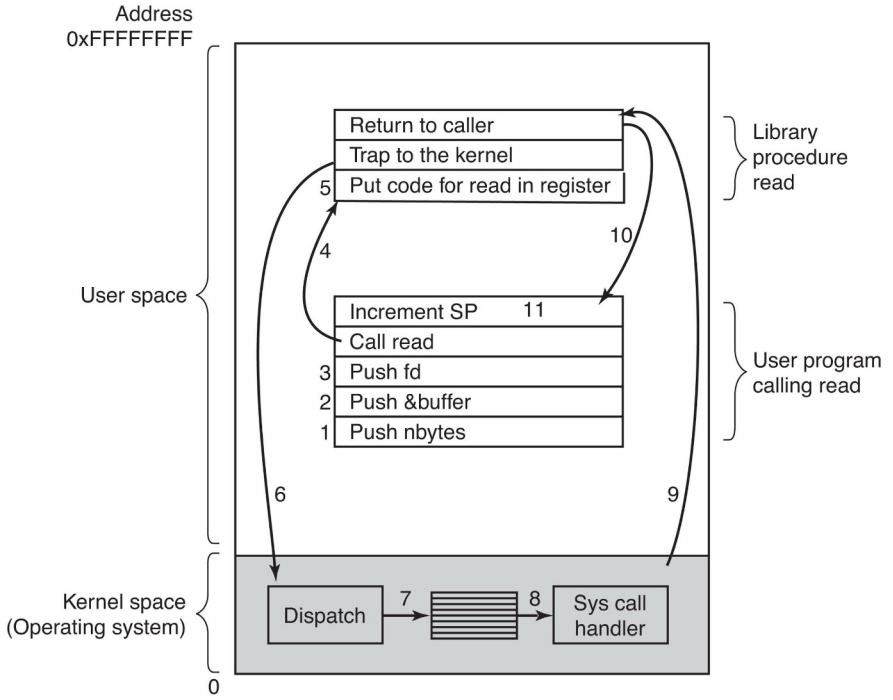


System Calls

- User Programmes interact with the OS with **System Calls**
  - E.g. Read from a file

```
read(fd, buffer, nbytes);
```

- System calls return **status codes**



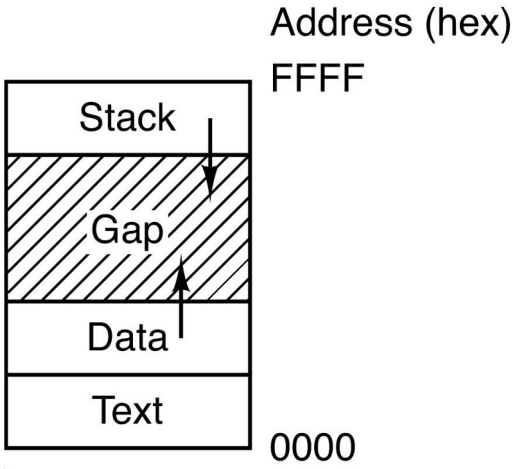
POSIX System Calls

- Portable Operating System Interface (POSIX)
  - A standardised Application Programming Interface (API) for maintaining **compatibility** between operating systems
  - POSIX-certified
    - macOS, Solaris
  - POSIX-compliant
    - Linux, Android, FreeBSD, Contiki
  - POSIX for Windows
    - Cygwin

Process

- A **process** is a program in execution
  - Associated with its **address space**
  - Other resources (e.g. registers, open files)
  - A process is a container that holds all the information needed to execute a program
- In a multiprogramming environment multiple processes run in **parallel** or **pseudo-parallel**
- The operating system maintains a **process table**: a list of all the processes (active, suspended, terminated until reaped)
- Processes have a unique Process ID (**PID**) and an **owner**
- What are the running processes?

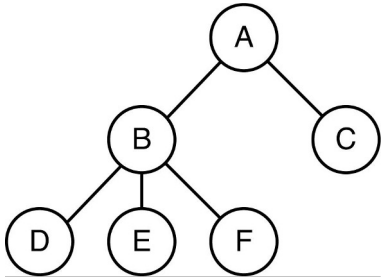
```
> ps -ef
```



Fork

- A process can **clone itself**, we call this **fork**
  - A process that forks is called the **parent process**, which creates a **child process**
  - The child process is an **exact copy** of the parent process
    - **Different address space** with exact **same contents**
  - Fork is the **only way** a process can be created
    - All processes are children of the **init** process

      ```
      > pstree
      ```
  - For the **parent** fork() returns the **child's PID**
  - For the **child** fork() returns **0**



- System Calls
  - ```
    pid_t fork(void); // creates a new process
    pid_t getpid(void); // returns PID of calling process
    ```
  - ```
    pid_t getppid(void); // returns PID of parent process
    ```
  - ```
    void exit(int status); // process terminates
    ```
    ```

Shared Memory

- After fork, parent and child process have **different address spaces**
  - They **cannot** exchange data via the global variables or heap
  - Yet, it is possible to ask the OS for some **shared memory**

- System Calls
  - `void *mmap(void *addr, size_t len, int prot, int flags, int fd, off_t offset); // allocates memory`

Wait

- The parent and child processes are executed in (pseudo-) parallel
- A parent process can **wait** for the child process to terminate
  - **Orphan process** is a child process whose parents has terminated
- The **wait** system call removes the child process from the process table
  - Parents must always wait for their children
  - **Zombie process** is a child process which has terminated but is still in the process table
- The parent can wait for **any** child to terminate or for a particular **PID**
- Through wait the parent has access to the **exit code** of the child

- System Calls
  - `pid_t waitpid(pid_t pid, int *status, int options);`
    `// waits for child process to terminate`

Signals

- Process can communicate with each other using **signals**
- There are different types of signals
  - SIGALRM, SIGINT, SIGTERM, SIGKILL … (`>kill -l`)
- Signals are software-based interrupts
  - They interrupt the normal execution to run a **handler function**
  - A **default signal handler** is executed unless the process has registered a **custom signal handler**
  - Some signals cannot be caught or ignored

- System Calls
  - `sighandler_t signal(int signum, sighandler_t handler); // registers a signal handler for a particular signal`
  - `unsigned int alarm(unsigned int seconds); // schedules an SIGLALRM signal`
  - `int kill(pid_t pid, int sig); // sends a signal to a process`

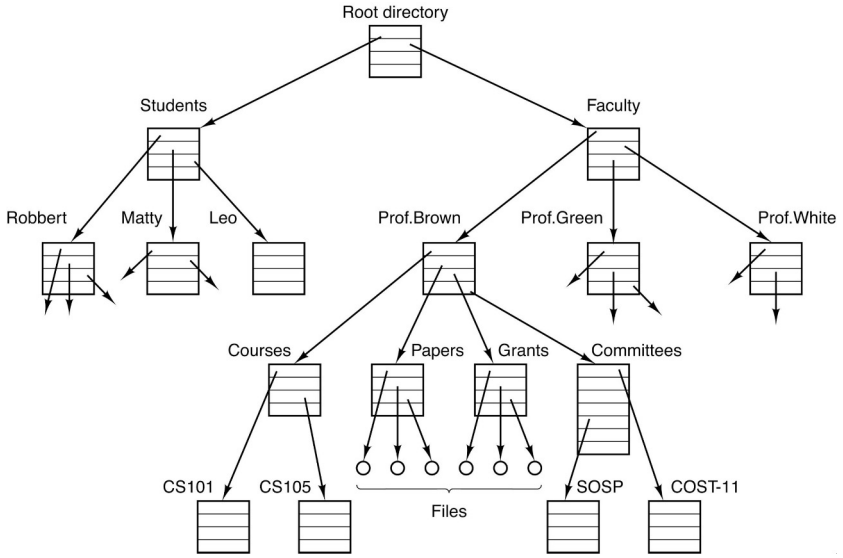
Exec

- The **exec** system call replaces the core image of the current process
- Remember: Only with **fork** we can create **new processes**
- New programmes can be executed in two steps:
  - The parent process calls **fork** to create a clone child process
  - The child process calls **exec** to replace its code with another executable

- System Calls
  - `int execl(const char *path, const char *arg, ...);`
    `//  Replaces the current process image with a new one`

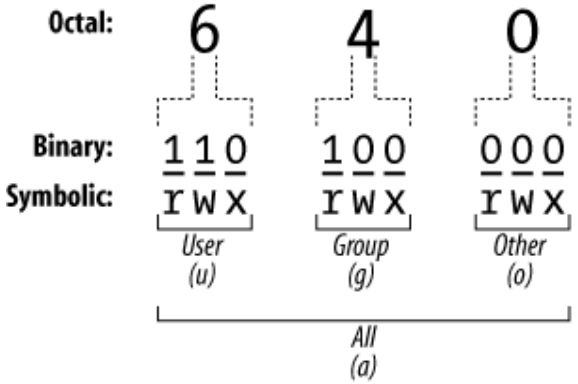
File

- A **file** is a data container in the **disk** (but not only that!)
- **Directories** group files together
- Files are organised in **trees**
- Each open file is assigned a **file descriptor**
  - A file descriptor is a **handle** to access files



File Permissions

- Access to files is restricted by the **file permissions**
  - File permissions have the form of **bitmap** of 9 digits
  - `$> ls -al`



- System Calls
  - `int chmod(const char *path, mode_t mode);`
    `// changes permissions of a file`

"Everything in UNIX is a file"

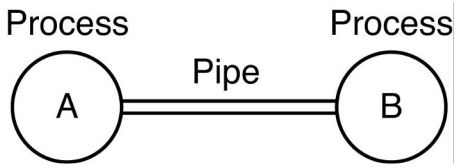
- UNIX has **special files**
  - Keyboard, screen
  - Hard drives, CD-ROMs, Serial Ports, Printers, other I/O peripherals
  - Various System and Hardware properties (e.g. CPU temperature)
  - Modems, Networking Interfaces

- They are not really files but they are exposed in the file system and get a **file descriptor**
  - "Everything in UNIX is a file descriptor"

- The same tools can be used!
  - E.g. the same system call can read data from an actual file, a serial port, or from the keyboard
  - Terminal redirection tools: >, <, |
  - Standard file descriptors: STDIN (0), STDOUT (1), STDERR (2)

Pipe

- Pipe is a **unidirectional data channel** (special file) that can be used for **interprocess communication**
  - Connects the **output** of one process to the **input** of another



- System Calls
  - `int pipe(int pipefd[2]); // creates a pipe`
  - `int dup2(int oldfd, int newfd); // creates a copy of a a file descriptor, using the new file descriptor`

Process management

Call	Description
pid = fork( )	Create a child process identical to the parent
pid = waitpid(pid, &statloc, options)	Wait for a child to terminate
s = execve(name, argv, environp)	Replace a process' core image
exit(status)	Terminate process execution and return status

File management

Call	Description
fd = open(file, how, ...)	Open a file for reading, writing, or both
s = close(fd)	Close an open file
n = read(fd, buffer, nbytes)	Read data from a file into a buffer
n = write(fd, buffer, nbytes)	Write data from a buffer into a file
position = lseek(fd, offset, whence)	Move the file pointer
s = stat(name, &buf)	Get a file's status information

Directory- and file-system management

Call	Description
s = mkdir(name, mode)	Create a new directory
s = rmdir(name)	Remove an empty directory
s = link(name1, name2)	Create a new entry, name2, pointing to name1
s = unlink(name)	Remove a directory entry
s = mount(special, name, flag)	Mount a file system
s = umount(special)	Unmount a file system

Miscellaneous

Call	Description
s = chdir(dirname)	Change the working directory
s = chmod(name, mode)	Change a file's protection bits
s = kill(pid, signal)	Send a signal to a process
seconds = time(&seconds)	Get the elapsed time since Jan. 1, 1970